Deep Q-Networks for 10×10 Battleship: Raw Boards versus Density Features

Le Cheng
Khoury College of Computer Sciences
Northeastern University, Boston, MA, USA
cheng.l1@northeastern.edu

***Abstract*—Battleship is a partially observable game in which an agent must sink a hidden fleet in as few shots as possible, and tabular Q-learning does not scale to the classic 10×10 board. We train Deep Q-Networks (DQN) with a fully convolutional dueling network, Double DQN targets and action masking, on 256 games in parallel; one 10×10 run of 100k games takes 12–14 minutes on a laptop. On 4×4, a DQN that sees only the board reaches 8.92 expected shots, statistically level with the probability-density heuristic (8.86) and 2% above the exact optimum (8.75). On 2,000 held-out 10×10 boards, the board-only DQN needs 53.00 shots. Adding the density map as an input plane brings it to 45.85,** 1.20 ± 0.46 **shots behind probability density (44.65). Using the density as a fixed baseline under a learned correction did not improve on it.**

I. Introduction

In single-player Battleship, ships of lengths 5, 4, 3, 3 and 2 are placed uniformly at random on a 10×10 grid (they may touch but not overlap), and each shot reveals *miss*, *hit* or *sunk*. The hidden layout makes this a POMDP [5]. The strongest simple strategy is *probability density* [6]: enumerate every position of every remaining ship that fits the board, weighting positions through open hits more, and fire at the cell covered by the largest total. It is greedy and treats ships independently, so a learner could, in principle, do better.

Our project group's tabular agents show why function approximation is needed. On 4×4, a Q-table keyed on the visible board visited 3.5 million states in a million games and stayed at random level (13.9 shots, against 14.1 for random on the same 40 test layouts). Keying the table on the exact belief, the set of layouts consistent with the board, reached 8.97, but on 10×10 there are 30,093,975,536 layouts, too many to store a belief. Battleship (with ships not allowed to touch) is a benchmark for online Monte-Carlo tree search [7]; the best published DQN we found reports about 52 shots [8].

We ask two questions. First, *can a DQN [1] learn 10×10 Battleship from the raw board alone*, without being told how ships are placed? Second, *does the probability-density map, which encodes that knowledge, help more as an input feature or as a starting policy?*

II. Method

MDP. The state s is the visible board plus the lengths of the ships still afloat. Observations are exact, so s determines the belief and is a Markov state. Actions are the unfired cells. Every shot has reward -1 and $\gamma = 1$, so the return is minus the game length and $Q(s, a)$ is minus the expected number of shots still needed, counting the shot at a.

Q-learning and DQN. Q-learning [2] moves $Q(s, a)$ toward the target $y = r + \gamma \max_{a'} Q(s', a')$. DQN replaces the table with a network Q_θ, trained with the Huber loss on $Q_\theta(s, a) - y$ over transitions sampled from a replay buffer, with y computed by a target network θ^- that is copied from θ periodically. We use Double DQN targets [3], $y = -1 + Q_{\theta^-}(s', \arg\max_{a'} Q_\theta(s', a'))$, and $y = -1$ when the shot ends the game. Fired cells are masked to $-\infty$ both when choosing a shot and inside the target.

Input and network. The board is encoded as 10×10 planes: hit, miss, sunk, an all-ones on-board plane (so the zero padding of the convolutions marks the edge), and five planes holding the number of ships afloat of each length 1–5. The network has four 3×3 convolutions with 32 channels and ReLU, giving a 9×9 receptive field: enough to see a 5-cell carrier passing through any cell. A dueling head [4] computes $Q = V(s) + A(s, a) - \overline{A}(s)$, where $\overline{A}$ is the mean of A over legal cells. The advantage A comes from a 1×1 convolution, a local quantity, and the value V from global mean and max pooling, because the shots still needed depend on the whole board. The network has 34.6k weights.

Variants. (A) *Board*: the planes above. (B) *Density*: A plus the probability-density map, scaled so the best cell is 1. It is computed from the board, so it adds no information, only a hand-crafted feature. (C) *Residual*: B with $Q = c\,\hat{d}(s, a) + Q_\theta(s, a)$, where $\hat{d}$ is the scaled density and $c = 1$. A's last layer starts at zero, so the untrained agent always fires at a highest-density cell, and DQN learns a correction [9].

Training. 256 games run in parallel as tensors, and each iteration fires one shot in every game. Each stored transition is trained on 8 times on average (batch 256), with a 200k-transition buffer, Adam (learning rate 3×10^{-4}) and a target copy every 1,000 gradient steps. Every run lasts 100k games. On 10×10, ε decays linearly from 1.0 (C: 0.1) to 0.02 over the first 10k games; on 4×4 over the first 30k. The games and the density run on the CPU, the network on the laptop GPU; unit tests check both against the reference code.

III. Experiments

Protocol. On 10×10, every 5k games we play the greedy policy on 500 validation boards (seeds 100,000–100,499) and keep the best checkpoint. For C the best was the untrained

TABLE I
MEAN SHOTS TO SINK THE FLEET (LOWER IS BETTER)

10×10, 2,000 test boards Agent	Shots	±95%	Paired vs. density	Train
Random	95.27	0.22	+50.62 ± 0.45	–
Hunt-target + parity	50.76	0.39	+6.11 ± 0.55	–
Probability density [6]	44.65	0.40	–	–
Belief greedy (particles)	44.50	0.41	−0.15 ± 0.51	–
DQN-A (board)	53.00	0.45	+8.35 ± 0.54	12.4 min
DQN-B (+ density input)	**45.85**	0.32	+1.20 ± 0.46	14.1 min
DQN-C (residual)	46.05	0.32	+1.41 ± 0.46	14.0 min
4×4, all 264 layouts				
Random*	14.23			
Q-table on visible board†	13.9			
Q-table on exact belief	8.97			7 min
DQN-A (board)	**8.92**		+0.06 ± 0.26	1.3 min
Probability density*	8.86		–	
Optimal V^* (exact)	8.75			

*One random tie-breaking per layout. †Measured by our group on 40 test layouts.

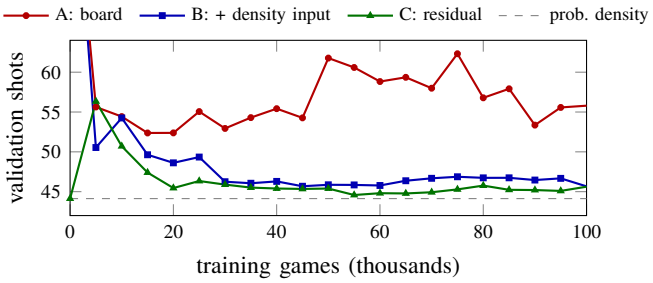


Fig. 1. Greedy score on the 500 validation boards during 10×10 training. Dashed: probability density on these boards (44.13). Untrained A and B start above the axis (90.5, 93.5).

network, which is just probability density, so we report C's final checkpoint. The test set is 2,000 other boards (seeds 0–1,999), the same for every agent, played through the project's standard engine, which shows agents only public information. We report means with 95% intervals (1.96 standard errors) and the *paired* difference to probability density, averaged board by board. On 4×4 every agent plays all 264 layouts, the exact expectation for the deterministic DQN; with no separate test set there, we report the final checkpoint.

Results. Table I and Fig. 1 give the results. On 4×4, DQN-A learned from the board in 75 seconds and is level with probability density, while the group's Q-table on the same input stayed at random level (trained and evaluated differently, so the comparison is only indicative). On 10×10, DQN-A fell from 90.5 to 52.4 validation shots within 15k games, comparable to the earlier DQN's ∼52 [8]. It then became unstable, ranging from 52 to 62, while its training loss rose from 0.36 to 1.9. Adding the density input (B) saved 7.15 shots over A on the test set and came within 1.20 ± 0.46 shots of probability density. The residual agent (C) started at the density level, but early training pushed it to 56 shots, and it never beat its start: its best trained score, 44.58 at 55k games, is still above density's 44.13. No DQN variant beat probability density, and neither did belief greedy, the project's particle-filter planner (−0.15 ± 0.51, not significant).

Where the shots go. We count a shot as *hunting* if no damaged, unsunk ship is on the board, and as *targeting* otherwise. Probability density spends 27.46 shots hunting and 17.19 targeting; DQN-B spends 17.09 and 28.76, most while a damaged ship is afloat. Shots elsewhere also count as targeting while a ship is unfinished, so this shows where DQN-B differs, not that it finds ships sooner.

IV. DISCUSSION AND CONCLUSION

From the raw board alone, a convolutional DQN learns 4×4 Battleship in about a minute and reaches 52–53 shots on 10×10 within 15k games (about two minutes). As an input, the density map is worth about 7 shots (B against A). As a starting policy it gave no gain: early, poorly calibrated corrections degraded it, and B and C end level (45.85 against 46.05), both behind density. The room for improvement is small: on 4×4, probability density is only 1.3% above the optimum. If the gap is similar on 10×10, that is about 0.6 shots, which a learner must gain without first losing more.

Limitations. One training seed per variant, no hyperparameter search, and checkpoint selection on a single set of 500 validation boards.

Future work. A lower learning rate and less frequent target copies to stop A's divergence; n-step returns and early stopping to need fewer games; several seeds per variant; and a residual agent that learns V before it may change the density's ranking. Code, tests, models and logs are in the project repository (`battleship/dqn`).

ACKNOWLEDGMENT

The code, experiments and analysis were developed with the assistance of Claude (Anthropic), an AI assistant. The author designed the study, directed the work and is responsible for the results. The tabular Q-learning baselines are from the author's project group.

REFERENCES

[1] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

[3] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI*, 2016.

[4] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," in *Proc. ICML*, 2016.

[5] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, pp. 99–134, 1998.

[6] N. Berry, "Battleship," DataGenetics blog, Dec. 2011, datagenetics.com/blog/december32011/.

[7] D. Silver and J. Veness, "Monte-Carlo planning in large POMDPs," in *Advances in Neural Information Processing Systems 23*, 2010, pp. 2164–2172.

[8] GA-Intelligence, "Deep reinforcement learning–of how to win at Battleship," blog post, ga-intelligence.com.

[9] T. Johannink *et al.*, "Residual reinforcement learning for robot control," in *Proc. ICRA*, 2019.